

Prova Técnica — Desenvolvimento de Sistemas

Front-end com Banco de Dados · Stack: HTML · CSS · JavaScript · Neon.tech

DURAÇÃO

3 horas

PONTUAÇÃO TOTAL

100 pontos

NOTA PARA APROVAÇÃO

60 pontos

CONSULTA

Somente MDN e Neon Docs

Nome: _____

Turma: _____

Nota: _____



CONTEXTO E MISSÃO

A startup inovadora **Nexus Corp** acaba de fechar uma rodada milionária de investimentos e sua equipe está dobrando de tamanho a cada mês. O antigo controle de acessos e funcionários, feito em planilhas, entrou em colapso.

Você foi escalado como o **Desenvolvedor Front-end** responsável por criar a **versão 1.0 do Portal de Gestão de Identidade** da empresa. O sistema precisa ser uma *Single Page Application (SPA)* limpa e responsiva, onde o RH possa **cadastrar** novos colaboradores, **listar** os existentes, **atualizar** seus dados e **excluí-los** — tudo em uma única tela.

A arquitetura do banco de dados (PostgreSQL) já foi provisionada na nuvem pela equipe de DevOps via **Neon.tech**. Sua missão é construir a interface e conectá-la ao banco via API HTTP.



LAYOUT DE REFERÊNCIA

◆ WIREFRAME ESPERADO — USE COMO GUIA DE ESTRUTURA (NÃO PRECISA SER IDÊNTICO)

Novo Usuário

Nome Completo *

E-mail *

Status



Salvar Usuário

Usuários Cadastrados

[Atualizar](#)

ID	NOME / E-MAIL	STATUS	AÇÕES
#1		Ativo	
#2		Inativo	
#3		Ativo	

1 Layout dividido em dois painéis: formulário (esquerda) + listagem com ações (direita). As badges de status devem ser coloridas: verde para Ativo, vermelho para Inativo.



PALETA DE CORES OFICIAL

Utilize **obrigatoriamente** as variáveis CSS abaixo. Declare-as em `:root {}` no início do seu CSS.



```

:root {
  /* Slate */
  --slate-50: #f8fafc;  --slate-100: #f1f5f9;  --slate-200: #e2e8f0;
  --slate-400: #94a3b8;  --slate-500: #64748b;  --slate-600: #475569;
  --slate-700: #334155;  --slate-800: #1e293b;
  /* Emerald / Green */
  --emerald-100: #d1fae5;  --emerald-600: #059669;  --emerald-700: #047857;
  --green-600: #16a34a;
  /* Indigo / Blue */
  --indigo-500: #6366f1;  --indigo-600: #4f46e5;  --indigo-700: #4338ca;
  --blue-50: #eff6ff;  --blue-600: #2563eb;
  /* Red / Rose */
  --red-50: #fef2f2;  --red-500: #ef4444;  --red-600: #dc2626;
  --rose-100: #ffe4e6;  --rose-600: #e11d48;  --rose-700: #be123c;
}

```



DADOS DO BANCO DE DADOS

O banco já está criado. O professor fornecerá o endpoint e o token antes do início. Preencha abaixo ao receber:

CAMPO	VALOR
Endpoint HTTP (Neon)	https://_____.neon.tech/sql/v1/query
Token (Bearer)	_____
Tabela	<code>usuarios</code>
Colunas	<code>id</code> (serial PK) · <code>nome</code> (varchar) · <code>email</code> (varchar) · <code>status</code> (varchar: <code>'Ativo'</code> ou <code>'Inativo'</code>)



1

Estrutura Visual — HTML e CSS

Sem frameworks CSS · Layout com Grid ou Flexbox · 4 arquivos

20 pontos

O projeto deve ser organizado em **4 arquivos separados**, cada um com responsabilidade própria:

ARQUIVO	RESPONSABILIDADE
<code>index.html</code>	Estrutura HTML da página, importação dos outros arquivos
<code>style.css</code>	Toda a estilização em CSS puro, usando as variáveis da paleta
<code>script_db.js</code>	Apenas a comunicação com o banco (fetch + queries SQL)
<code>script.js</code>	Toda a lógica de interface (manipulação do DOM, eventos, toasts)

O `script.js` deve ser carregado no HTML como **módulo ES6** (`type="module"`) e importar as funções do `script_db.js` via `import { ... } from './script_db.js'`. Utilize a biblioteca **Font Awesome 6** (CDN) para os ícones dos botões.

Layout: dois painéis lado a lado com CSS Grid (`1fr 2fr` no desktop). O formulário deve ter `position: sticky` para acompanhar o scroll da tabela.

Formulário deve conter:

- Título dinâmico (muda entre "Novo Usuário" e "Editar Usuário")
- Botão **"Cancelar Edição"** — visível apenas no modo edição (classe `hidden` para ocultar)
- Campo oculto `<input type="hidden" id="user-id">` — armazena o ID durante o UPDATE
- Campos: *Nome Completo* (texto, obrigatório), *E-mail* (email, obrigatório), *Status* (select: "Ativo" / "Inativo")
- Botão de submit com ícone `fa-save` e texto dinâmico

Tabela deve ter cabeçalho: ID · Nome/E-mail · Status · Ações (centralizado). Botão "🔄 Atualizar" no header do card da tabela, chamando a função de listagem manualmente.

🚫 **Proibido:** frameworks CSS (Bootstrap, Tailwind, etc.) e frameworks JS (React, Vue, etc.). CSS3 e JavaScript puro.

2

Módulo de Banco — script_db.js e SELECT

25 pontos

Motor central de fetch · Exportação de funções · Leitura de dados

No `script_db.js`, declare a **connection string** do Neon e construa o endpoint HTTP extraindo o host dela:

```
const DATABASE_URL = "postgresql://usuario:senha@host/banco?sslmode=require";
const host = new URL(DATABASE_URL).host;
const neonHttpEndpoint = `https://${host}/sql`;
```

Crie a função **auxiliar central** `executarQueryNeon(querySQL, parametros)` que encapsula todo o `fetch()`.

Todas as outras funções CRUD devem chamá-la — nunca repetir o fetch diretamente. O header de autenticação obrigatório é:

```
'Neon-Connection-String': DATABASE_URL
```

A função deve retornar `dados.rows` em caso de sucesso, ou `null` em caso de erro (use `try/catch`). Erros devem ser exibidos no `console.error`.

Implemente e **exporte** a função `consultarDiretoComFetch()` que:

- Executa `SELECT * FROM usuarios` via `executarQueryNeon()`.
- Retorna o array de usuários, ou `[]` se ocorrer erro (nunca `null` para a tela).

No `script.js`, implemente `criarTabelaUsuarios()` que:

- Chama `consultarDiretoComFetch()` e limpa o `tbody` antes de renderizar.
- Gera cada `<tr>` dinamicamente com: célula de ID (prefixo `#`), célula Nome + e-mail em linhas separadas, badge de status e coluna de ações.
- Badges: classe `badge-active` (fundo `--emerald-100`, texto `--emerald-700`) para "Ativo" e `badge-inactive` (fundo `--rose-100`, texto `--rose-700`) para "Inativo".
- Se não houver registros, exibir linha com `colspan="4"` e mensagem de estado vazio.
- Ser chamada **automaticamente ao carregar a página**.

💡 Os registros retornados pelo Neon estão em `dados.rows`. O endpoint HTTP aceita apenas o método `POST` com `Content-Type: application/json`.

3

Cadastro de Usuários — INSERT e UPDATE

25 pontos

Função unificada de envio · Criação e atualização no mesmo formulário

No `script_db.js`, implemente e exporte as funções de escrita:

- `insertUsuario(nome, email, status)` — usa `INSERT INTO ... RETURNING *` e retorna o objeto do usuário criado (ou `null` em erro).
- `sqlAtualizarUsuario(id, nome, email, status)` — usa `UPDATE ... SET ... WHERE id = $4 RETURNING *`. **A ordem dos parâmetros importa:** `[nome, email, status, id]`.

```
// INSERT
"INSERT INTO usuarios (nome, email, status) VALUES ($1, $2, $3) RETURNING *"
params: [nome, email, status]

// UPDATE
```

```
"UPDATE usuarios SET nome = $1, email = $2, status = $3 WHERE id = $4 RETURNING *"  
params: [nome, email, status, id]
```

No `script.js`, crie a função **unificada** `lidarComEnvioDoFormulario(event)` vinculada ao evento `submit` do formulário:

- Chame `event.preventDefault()` para evitar recarregamento.
- Leia o valor do campo oculto `user-id`: se estiver preenchido → é um **UPDATE**; se estiver vazio → é um **INSERT**.
- Após operação bem-sucedida: chamar `limparFormulario()` e `criarTabelaUsuarios()`.
- Em caso de erro: exibir toast de erro e **não limpar** o formulário (permite tentar novamente).

Implemente também `limparFormulario()` que: chama `form.reset()`, limpa o campo oculto `user-id`, restaura o título e o texto do botão, e esconde o botão "Cancelar Edição".

💡 O campo oculto é a chave: `document.getElementById('user-id').value` vazio = CREATE; preenchido = UPDATE. Essa lógica mantém o formulário em modo de edição até o usuário salvar ou cancelar.

4

UX, DELETE e Modo de Edição

Toasts · Exclusão com confirmação · prepararEdicao() · window scope

30 pontos

Toasts (notificações flutuantes):

- Crie a função `mostrarToast(mensagem, tipo)` — o parâmetro `tipo` aceita três valores: `'success'`, `'error'` e `'info'`.
- Cada tipo deve ter cor e ícone (Font Awesome) diferentes:
 - `success` → fundo `--green-600`, ícone `fa-check-circle`
 - `error` → fundo `--rose-600`, ícone `fa-exclamation-triangle`

- `info` → fundo `--blue-600`, ícone `fa-info-circle`
- O toast deve ser adicionado ao `#toast-container` (fixado no canto inferior direito da tela).
- Deve sumir automaticamente após **3 segundos** com animação de saída (opacidade + translate).
- No CSS, implemente a animação de entrada via `@keyframes slideIn` e aplique com a classe `toast-enter`.

Função de Exclusão — DELETE:

- No `script_db.js`: exporte `sqlDeletarUsuario(id)` com query `DELETE FROM usuarios WHERE id = $1 RETURNING *`. Retorna o objeto deletado ou `null`.
- No `script.js`: declare `window.deletarUsuario = async function(id)` — o `window.` é obrigatório pois a função é chamada via `onclick` inline no HTML gerado dinamicamente.
- Exibir `confirm()` antes de excluir. Se o usuário cancelar, não fazer nada.
- Após exclusão bem-sucedida: toast de sucesso + recarregar tabela.

Função de Edição — prepararEdicao():

- Declare `window.prepararEdicao = function(id, nome, email, status)` — também em `window` pelo mesmo motivo.
- Deve preencher o campo oculto `user-id` com o ID recebido (isso ativa o modo UPDATE na função de envio).
- Preencher todos os campos do formulário com os dados da linha clicada.
- Atualizar o título para "Editar Usuário" e o texto do botão para "Atualizar Usuário".
- Exibir o botão "Cancelar Edição" (remover classe `hidden`).

Botões de ação nas linhas da tabela:

- Botão editar: fundo `--blue-50`, cor `--blue-600`, ícone `fa-pen`. No hover: fundo `--blue-600`, texto branco.
- Botão excluir: fundo `--red-50`, cor `--red-600`, ícone `fa-trash`. No hover: fundo `--red-600`, texto branco.
- Ambos os botões circulares (`border-radius: 9999px`), tamanho `2rem × 2rem`.

- Os `onclick` devem passar todos os dados da linha como argumentos: `onclick="prepararEdicao(id, 'nome', 'email', 'status')"`.

💡 Funções chamadas via `onclick=""` em HTML gerado pelo JS precisam estar no escopo global (`window.nomeDaFuncao`). Funções internas do módulo ES6 não são acessíveis por `onclick` sem isso.



CRITÉRIOS DE AVALIAÇÃO

CRITÉRIO	PESO	NOTA OBTIDA
TAREFA 1 — ESTRUTURA VISUAL (HTML + CSS)		
Projeto separado em 4 arquivos (index.html, style.css, script.js, script_db.js) com type="module" e FontAwesome importados	5 pts	_____
Layout em CSS Grid (1fr 2fr no desktop), formulário com position: sticky, responsivo em mobile	5 pts	_____
Formulário com campo oculto user-id, botão "Cancelar Edição" (oculto por padrão), título e texto do botão dinâmicos	5 pts	_____
Estilização com variáveis CSS da paleta, sem frameworks, hover nas linhas da tabela e nos botões de ação	5 pts	_____
TAREFA 2 — MÓDULO DE BANCO E SELECT		
Função central executarQueryNeon() com Neon-Connection-String no header, try/catch, retorna dados.rows ou null	8 pts	_____

CRITÉRIO	PESO	NOTA OBTIDA
Endpoint construído via <code>new URL(DATABASE_URL).host</code> — sem hardcode do host	4 pts	_____
Função <code>consultarDiretoComFetch()</code> exportada; <code>criarTabelaUsuarios()</code> renderiza badges coloridas, estado vazio e é chamada ao carregar	8 pts	_____
Funções CRUD exportadas no <code>script_db.js</code> e importadas no <code>script.js</code> com sintaxe ES6 (<code>import { ... } from</code>)	5 pts	_____
TAREFA 3 — INSERT E UPDATE (FUNÇÃO UNIFICADA)		
<code>insertUsuario()</code> com query parametrizada (<code>\$1, \$2, \$3</code>) e <code>RETURNING *</code> funcionando no banco real	8 pts	_____
<code>sqlAtualizarUsuario()</code> com <code>UPDATE ... RETURNING *</code> , parâmetros na ordem correta (<code>[nome, email, status, id]</code>)	7 pts	_____
Função <code>lidarComEnvioDoFormulario()</code> com <code>preventDefault()</code> , lógica de bifurcação pelo <code>user-id</code> , limpeza e reload da tabela após sucesso	5 pts	_____
TAREFA 4 — UX, DELETE E MODO DE EDIÇÃO		
<code>mostrarToast(mensagem, tipo)</code> com 3 tipos (success/error/info), cores corretas, some em 3s com animação CSS (<code>@keyframes slideIn</code>)	8 pts	_____
<code>window.deletarUsuario()</code> com <code>confirm()</code> , DELETE parametrizado com <code>RETURNING *</code> , toast + reload após sucesso	8 pts	_____
<code>window.prepararEdicao()</code> preenchendo todos os campos, o campo oculto, título, texto do botão e mostrando o botão cancelar	9 pts	_____

CRITÉRIO	PESO	NOTA OBTIDA
Botões circulares de editar/excluir em cada linha com hover invertido (cor passa para branco), ambos declarados em window	5 pts	_____
TOTAL	100 pts	_____



O QUE ENTREGAR

- ✓ Os 4 arquivos do projeto: `index.html`, `style.css`, `script.js` e `script_db.js` — compactados em **.zip** ou em repositório público no GitHub.
- ✓ A aplicação será testada abrindo o `index.html` diretamente no navegador (sem servidor local). Ela deve comunicar-se com o banco Neon via a connection string configurada no `script_db.js`.
- ✓ Print ou gravação de tela demonstrando as 4 operações: **listagem ao abrir**, **INSERT** de novo usuário, **UPDATE** via botão editar, e **DELETE** com confirmação.
- ✓ Os toasts devem aparecer visíveis na gravação para cada operação realizada.

⚠ **Nota zero nas tarefas 2, 3 e 4** para projetos que utilizem `localStorage`, arrays estáticos como mock permanente, ou que não executem requisições `fetch` reais ao banco Neon.

Dúvidas sobre `fetch` ou configurações do Neon → pergunte ao professor/avaliador.
Lógica de programação e montagem das queries SQL são de inteira responsabilidade do aluno.

Bom código! 🚀